

SpaceX 엔지니어들의 Grok Bot 가이드

72시간의 실시간 스트리밍, 3일간의 기록.



01/24

TABLE OF CONTENTS

가이드 목차 및 개요

01	실 험	3일간의 실시간 스트리밍이 실제로 만들어낸 결과물
02	멘탈 모델	태스크가 아니라 동료(팀원)로 대하기 — 관점의 변화가 설정을 바꾼다
03	봇이 실제로 부여받는 환경	자체 컴퓨터, 독립된 메모리, 그리고 공유 파일시스템
04	첫 번째 팀 구성하기	하나의 봇, 하나의 역할 — 그리고 봇 증식의 함정 피하기
05	봇을 만드는 봇	닥터 에그봇(Dr. Eggbot), 영혼으로서의 디스크립션, 과적합된 역할 수정
06	스킬과 루틴	시연을 통한 교육과 오류 교정의 추적
07	소프트웨어 팩토리	포테이토 모드, 에이전트 스위치, 그리고 완전 자율주행
08	검증이 성패의 전부다	가장 먼저 구축해야 할 최고의 레버리지
09	검증된 7가지 프롬프트 패턴	실전에서 거듭 확인된 핵심 프롬프트 기법
10	실전 프롬프트 라이브러리	생방송 현장에서 그대로 건져 올린 프롬프트 원문
11	확장 가능한 오케스트레이션	비서실장, 플레이북 전파, 스태프 미팅, 하위 봇 군단
12	실제 소요 비용과 경제성	실제 비용 데이터와 돈이 새어 나가는 4대 구멍
13	승인 절차와 위험 반경 통제	자동 리뷰, 보안 금고(Vault), 인젝션 방어, 서버 권한
14	케이스 스터디: Thursday Arena	72시간 만에 제로에서 실제 게임 출시까지
15	실제 투입된 봇 팀 명단	실제 팀에 투입된 모든 봇들의 명단과 담당 역할
16	실무 현장의 6가지 봇 조합	실제 업무 현장에서 가동 중인 6가지 실전 구성
17	라이브 방송 중 발생한 실패	방송 중에 터진 10가지 사고들과 그로부터 얻은 교훈
18	현재의 한계와 향후 로드맵	아직 지원되지 않는 기능들과 앞으로 출시될 기능들
19	팀의 솔직한 회고	72시간의 끝에서 엔지니어들이 털어놓은 낱것의 진실
20	첫 주 실천 가이드	바로 오늘 시작할 수 있는 실천 체크리스트

01 · THE EXPERIMENT

3일간의 실시간 스트리밍, 하나의 회사를 세우다

Grok Bot 팀의 세 엔지니어 — **맷 팔머(Matt Palmer)**, **로산(Roshan)**, 그리고 **로렌(Lauren, 일명 "Potato")** — 가 빈 GitHub 조직, 빈 Slack, 빈 Notion, 그리고 아무런 사업 아이디어도 없는 상태로 샌프란시스코의 한 스튜디오에 모였습니다. 그리고 72시간 후, 수천 명의 사용자가 플레이하는 실제 라이브 프로덕트가 탄생했습니다. 이 가이드의 모든 내용은 그 3일간의 실전 스트리밍에서 도출되었습니다.

Day 1 — 아이디어 도출

레스토랑 팝업 기획에서 팝업 플랫폼으로, 다시 아트 전시회로, 그리고 굿즈 팝업으로 하루 만에 네 번 피벗. 소프트웨어 구현은 결코 병목이 아니었습니다. 무엇을 만들지에 합의하는 것이 진짜 병목이었습니다.

Day 2 — 본격 개발

게임 스튜디오로 전격 피벗. 'Cupcake'이라는 코드명의 오토배틀러 게임을 순수 HTML 프로토타입으로 제작해 실시간으로 플레이 — 스텟 버그, 실시간 치팅 시연까지 날것 그대로 진행되었습니다.

Day 3 — 공식 출시

밤사이에 봇 팩토리가 170개의 PR을 머지했습니다. 오전 10시, 게임은 **Thursday Arena**라는 이름으로 공식 런칭되었습니다. 방송 종료 시점: 머지된 PR 433개, 페이지뷰 약 30,000회, 경기 수 약 6,000회, 매출 \$0.

433

3일간 머지된 풀 리퀘스트(PR) 수

~2,000

출시 당일 로그인한 실제 플레이어 수

~30k

출시된 게임의 총 페이지뷰 수

\$0

실제 매출 — 가감 없는 솔직한 수치



왜 라이브 스트리밍 기반 가이드가 읽을 가치가 있는가

공식 프로덕트 문서는 기능이 무엇을 하는지만 설명합니다. 하지만 편집되지 않은 72시간의 라이브 개발 과정은 사람들이 극심한 시간 압박 속에서 **실제로 무엇을 하는지** 보여줍니다 — 어떤 패턴이 끝까지 살아남고, 어떤 패턴이 과도한 비용을 초래하며, 어떤 패턴이 조용히 망가지는지를 말이죠. 여기에 정리된 거의 모든 내용은 누군가의 설명이 아닌, 실제 행동에서 추출한 실전 프랙티스입니다.

*이름 표기에 대한 참고: 자동 자막 트랜스크립트에서는 이 제품을 "GrokBot", "Rockbot", "Brockbot"으로, 회사를 "SpaceX AI" 등으로 표기하기도 하지만 모두 동일한 대상을 의미합니다. 수치는 스트리밍 중에 직접 언급된 것으로 당일 기준입니다.

02 · MENTAL MODEL

태스크가 아니라 동료(팀원)로 대하기

제품 기획자 중 한 명인 로만(Roman)은 첫 번째 스트리밍의 문을 열어 전체 제품을 관통하는 핵심 명제를 제시했습니다: **"우리는 사용자가 Grok Bot을 쓸 때, 마치 실제 직장 동료나 팀원과 함께 일하는 것처럼 느끼기를 진심으로 바랍니다."** 인터페이스가 의도적으로 iMessage 형태로 디자인된 이유도 바로 여기에 있으며, 그 형태가 이 가이드의 모든 모범 사례를 이끕니다.

제품을 지탱하는 3가지 기본 전제

- 1 태스크 패러다임이 아닌, 동료(Teammate) 패러다임.** 봇은 한번 쓰고 버리는 일회성 대화 스레드가 아닙니다. 몇 달 동안 계속해서 다시 찾아가는, 이름이 있는 동료입니다. 지식과 맥락이 계속 축적됩니다.
- 2 모든 봇에게 자체 컴퓨터가 주어진다.** 브라우저가 탑재된 독립된 Linux 가상머신(VM)입니다. 레거시 소프트웨어, 정부 포털, API가 전혀 없는 사이트 등 마우스와 화면으로 사람이 할 수 있는 모든 작업을 수행할 수 있습니다.
- 3 모든 것이 클라우드에서 실행된다.** 노트북 덮개를 닫아도 작업은 중단되지 않습니다. "당신이 휴가를 떠나 있거나 잠든 밤 사이에도 봇은 묵묵히 일을 계속합니다."

실무에서 달라지는 점

- **설정하는 것이 아니라, 채용하는 것이다.** 봇의 범위를 정의하는 일은 설정 페이지를 채우는 것보다 채용 직무 기술서(JD)를 작성하는 것에 훨씬 가깝습니다.
- **피드백은 복리로 누적된다.** 봇이 왜 틀렸는지 이유를 가르쳐주는 것은 귀찮은 방해가 아니라 미래를 위한 투자입니다.
- **완성된 작업물로 돌아온다.** 암리타(Amrta)의 명언: "이미 완성된 작업물을 확인하는 순간이야말로 Grok Bot 최고의 경험입니다."
- **중간에 개입해도 괜찮다.** 봇이 작업 중일 때 방향을 수정해 주는 것은 지극히 정상이며 적극 권장됩니다.

모델(Model) vs 하네스(Harness)

로산(Roshan)은 작업이 어디서 이루어져야 하는지 명확한 경계선을 그었습니다:

- **Grok**은 파운데이션 모델이다.
- **Grok Bot**은 하네스(Harness)다 — 의도적으로 가볍게 설계되어 오케스트레이션, 도구 활용, 의사결정 및 판단에 최적화되어 있습니다.
- **Cursor 클라우드 에이전트**는 무거운 코딩을 담당하는 하네스다 — 실제 소프트웨어 엔지니어링 작업이 필요할 때 Grok Bot이 작업을 위임하는 대상입니다.

Grok Bot 안에서 프로토타입을 만들고, 클라우드 에이전트를 통해 배포하십시오.

“90% 수준까지 도달하는 것과, 사람이 손 하나 대지 않고 처음부터 끝까지 완전하게 끝내는 것의 차이는 정말 엄청납니다. 실로 마법 같은 경험이지요.”

로만(Roman) · Grok Bot 101, Day 1

03 · ARCHITECTURE

봇이 실제로 부여받는 환경

여러분들이 만든 봇들 사이에 무엇이 공유되고 무엇이 공유되지 않는지 명확히 아는 것은, 스트리밍 참가자들이 겪은 수많은 놀라운 현상 (긍정적이든 부정적이든)을 이해하는 열쇠입니다.

🖥️ 자체 컴퓨터 (Linux VM)

- 봇마다 브라우저, 터미널, 설치 가능한 앱이 포함된 전용 **Linux VM**이 제공됩니다.
- 한 봇이 다른 봇의 화면을 들여다볼 수 없습니다 — 이 격리 덕분에 두 봇이 같은 슬라이드 덱의 서로 다른 페이지를 동시에 편집할 수 있습니다.
- 로그인, 2FA 인증, 캡차(CAPTCHA) 등 직접 통제하고 싶은 순간에는 사용자가 화면을 가로채 직접 조작할 수 있습니다.
- VM이 Linux 기반이므로, **Linux와 호환되지 않거나 MCP로 제공되지 않는 도구는 아예 사용할 수 없습니다.** (Q&A에서 명확히 밝혀짐).

🧠 독립된 메모리

- 메모리는 **봇마다 개별적으로** 부여되며, 오래 유지되고 직접 편집할 수 있습니다. 한 번 지정한 사용자 선호도는 영구히 유지됩니다.
- 각 봇은 고유한 **컨텍스트 한계(Context Limit)**를 가집니다 — 업무를 여러 세부 역할로 쪼개야 하는 가장 현실적인 이유입니다.
- 필요 없는 맥락은 잊어버리라고(Forget) 명령할 수 있으며, 이를 통해 토큰 비용을 절감합니다.
- **봇을 복제(Duplicate)하면** 페르소나는 복사되지만 메모리는 0에서 새로 시작합니다.

📁 공유 파일시스템

- 봇들은 컨텍스트는 공유하지 않지만 **파일은 공유합니다** — "하나의 VM 위에 여러 개의 서로 다른 데스크톱 창을 띄워둔 것과 같은 구조입니다."
- **스킬은 전역(Global)으로 적용됩니다.** 한 봇에게 가르쳐준 스킬은 조직 내의 모든 봇이 즉시 사용할 수 있습니다.
- 브라우저 로그인 세션은 머신 상에 저장되어 서로 다른 작업 사이에서도 유지됩니다.

🔧 도구에 접근하는 두 가지 방식

- **커넥터 / MCP** — "에이전트를 위한 API." 훨씬 빠르고 저렴하며 권한 허용 목록(화이트리스트) 관리가 쉽습니다. 존재하는 경우 무조건 우선 사용하십시오.
- **컴퓨터 유즈 (Computer Use)** — API가 없는 모든 작업을 위한 보편적 대체재입니다. 단, 속도가 느리고 비용이 더 많이 듭니다.
- 봇에게 MCP URL을 알려주면 **알아서 커넥터를 직접 설치합니다.**



공유 파일시스템은 보안 경계가 아닙니다

메모리는 분리되어 있지만 파일과 브라우저 세션은 공유되므로, 봇을 나눈다고 해서 권한(Access)이 분리되는 것이 아니라 단지 **관심사(Attention)가 분리될 뿐**입니다. 진정한 보안 경계가 필요하다면 다른 봇을 만들 것이 아니라 로그아웃, 커넥터 범위 제한, 계정 권한 분리를 통해 명시적 경계를 그어야 합니다.

*3일간 확인된 지원 플랫폼: macOS, Windows, Linux, iOS, iPadOS, Android. 설정에는 클라우드 VM 대신 로컬 머신에서 봇이 작동하도록 하는 로컬 실행 토글이 있지만, 팀은 병렬 처리 능력과 로컬 앱의 포커스 탈취 문제를 고려해 클라우드 환경을 강력히 권장합니다.

04 · BUILDING A ROSTER

하나의 봇, 하나의 역할

이것은 3일간 모든 발표자가 입을 모아 가장 많이 반복했던 단 하나의 조언입니다. 모든 것을 다 하는 만능 비서 하나가 아니라, 각자의 고유한 이름이 있는 **세부 전문가들로 구성된 명단(Roster)**을 운영하라는 것입니다.

단일 역할 봇이 훨씬 뛰어난 4가지 이유

- 1 컨텍스트 범위가 좁게 유지된다.** 각 봇마다 고유한 컨텍스트 한도가 있습니다. 4가지 무관한 작업을 동시에 저글링하는 제너럴리스트 봇은 금세 토큰을 소진하고 이전 내용을 망각하기 시작합니다.
- 2 누구에게 물어볼지 뇌가 쉽게 기억한다.** 링 시(Ling Shi)의 비유: "사람의 뇌도 소셜책 전체를 외울 수 없습니다. 정확히 누구를 참조해야 할지 아는 것이 핵심입니다."
- 3 명확한 역할이 주어질 때 모델의 성능이 극대화된다.** 피드백이 실질적인 의미를 가지려면 업무 범위가 충분히 좁아야 하며, 이때 학습 루프가 날카롭게 정렬됩니다.
- 4 공짜로 병렬 처리를 얻는다.** 5명의 전문가 봇에게 동시에 작업을 던져두고, 각자 일하게 한 뒤 나중에 결과를 취합하면 됩니다 — 사람이 일하는 '회의 → 분과별 실행 → 재동기화' 패턴과 완벽히 일치합니다.

좋은 역할 정의에 담겨야 할 내용

- 모호한 "일반 보조"를 피하고 좁게 정의하십시오: 인재 발굴(Talent Scout), 경비 관리자(Expense Manager), 버그 재현 전문가(Bug Reproduction).
- 이름 붙일 수 있을 만큼 명확하고 단일한 책임 영역.
- 해당 봇이 사용 가능한 구체적인 도구 및 데이터 소스.
- 단순히 어떤 일인지가 아니라 업무에 접근하는 사고방식.
- 작업을 실행하기 전에 반드시 사용자 승인이 필요한 항목.
- 정기적으로 반복해야 하는 업무인 경우 그 일정.

사이드바를 체계적으로 정리하기

- 조직도처럼 섹션별로 봇을 묶으십시오 — 리더십(Leadership), 엔지니어링(Engineering), 워룸(War Room).
- 라벨은 각 봇의 전문 분야를 시각적으로 상기시켜 주는 필수 요소입니다. 특히 음식 이름으로 봇을 부를 때는 더욱 중요합니다.
- 사이먼(Simon)은 기능별로 색상을 지정했습니다: 리서치(주황), 고객 어카운트(초록), 아웃바운드(기타) — 스투드 색상만 보고도 파이프라인의 어느 단계인지 즉시 파악할 수 있습니다.
- 매일 실제로 사용하는 핵심 봇 3~4개를 상단에 고정하십시오.

*이름 짓기는 단순한 장식이 아닙니다. 팀은 시청자 채팅을 통해 테이터(Tater), 위스크(Whisk), 베이크(Bake), 닥터 에그봇(Dr. Eggbot) 같은 이름을 붙였고, 이는 작업에 대해 소통하는 방식을 완전히 바꾸어 놓았습니다. 이름을 가진 봇에게는 자연스럽게 일을 위임하지만, '태스크 3' 같은 봇은 미세 관리(마이크로매니징)하게 됩니다.



봇 증식(Bot Sprawl)의 함정

스트리밍에서 가장 거대한 봇 팀을 운영했던 사이먼(Simon)은 솔직하게 털어놓았습니다: "저도 한때 봇을 너무 많이 만들었던 적이 있습니다. 솔직히 말해서 훨씬 더 혼란스러웠습니다. '새로운 봇이 이 일을 해야 할 진짜 이유가 있는가?'를 매번 의심해야 합니다." 블레이크(Blake) 역시 동일하게 조언했습니다: "팀을 슬림하게 유지하십시오. 45개의 봇은 절대 필요 없습니다." 봇을 새로 생성하는 건 쉽고 재미있기에 더욱 규칙이 필요합니다. 새 직원을 뽑기 전에, 기존 전문가에게 새로운 스킬이나 루틴을 추가할 수 있는지 먼저 검토하십시오.

05 · THE META-BOT

봇을 만드는 봇

3일간 가장 많이 호출된 봇은 엔지니어도, 리서처도 아니었습니다. 바로 다른 봇들을 생성하고, 검토하고, 개선하는 일만을 전담하는 메타 봇 **닥터 에그봇(Dr. Eggbot)**이었습니다. 스트리밍에 등장한 거의 모든 새로운 동료 봇은 에그봇에게 던진 한 문장에서 태어났습니다.

메타 봇이 수행하는 핵심 업무

- 일상적인 자연어 설명으로부터 새로운 봇을 생성.
- 시스템 프롬프트 역할을 하는 디스크립션(Description)을 작성.
- 팀 명단을 감사(Audit): "우리 봇들을 전체 검토해 봐. 어디가 병목이지?"
- 찰떡같은 작명. 두 사람이 서로 다른 방에서 따로 요청했는데 둘 다 "Ping"이라는 이름을 추천받았을 정도입니다.

디스크립션은 봇의 영혼이다

- 디스크립션 필드는 단순한 메모 라벨이 아닙니다. 로렌(Lauren)의 표현대로 "누군가는 영혼(Soul) 또는 시스템 프롬프트라고 부르는 것"입니다.
- 봇의 말투, 의사결정 기준, 하위 작업 위임 방식이 완전히 달라집니다.
- 특정 사고 사례가 아닌 일반적인 **원칙**으로 작성하십시오.
- 봇이 무엇을 소유하고 무엇을 다른 이에게 넘겨야 하는지 명시하십시오.
- 신뢰할 수 있는 단일 출처(Source of Truth)를 명시적으로 지정하십시오.

과적합(Overfitting)의 실패 — 그리고 올바른 해결책

닥터 에그봇이 처음에 작성했던 디스크립션

"우리 게임 스튜디오의 전체적인 맥락 파악. 단 하나의 업무: Pstack의 포테이토 모드와 클라우드 에이전트를 통해 개발 작업을 오케스트레이션하고, 이를 감독 및 검증하여 엔지니어링 결과물을 책임진다. 플레이북과 일치시키고 준수할 것."

→ 로렌의 평가: **지나치게 구체적이다**. 오늘의 톨체인과 오늘 발생한 특정 사건을 봇의 영구적인 정체성으로 하드코딩해 버렸다.

교정 지시 (THE CORRECTION)

"포테이토 모드를 다시 정독하고, 지나치게 지엽적인 이슈들에 매몰되는 대신 Cupcake 엔지니어가 따라야 할 일반 원칙들을 도출해 내라."

→ 이것은 봇 디스크립션뿐만 아니라 당신이 작성하는 모든 스킬과 규칙에 적용되는 대원칙입니다.

*적합한 것이 있다면 마켓플레이스 템플릿에서 시작하십시오. 템플릿은 메모리, 루틴, 연동 설정을 담고 있으며, 공유 시 자격증명과 대화 내역은 안전하게 제거됩니다. 팀 역시 이미 존재하는 템플릿이 있다면 바닥부터 만들지 말고 거기서 시작하라고 권장했습니다.



요청하면 봇들이 스스로 팀원을 채용한다

둘째 날 암리타(Amrta)는 두 개의 기존 봇에게 물었습니다: "너희가 지금 하고 있는 일을 바탕으로 볼 때, 계속해서 훌륭한 성과를 내기 위해 어떤 봇이 더 있으면 도움이 되겠니? 필요한 봇들을 직접 생성해 줘." 그러자 봇들은 즉시 세 명의 새로운 전문가 — 배틀카드 블레어(Battle Card Blair), 데모 드레이크(Demo Drake), AI 레이더(AI Radar) — 를 만들어냈으며, 각각 고유한 업무 정의와 권위 있는 데이터 출처까지 완비했습니다. 봇 명단에게 무엇이 부족한지 직접 묻는 것은 대단히 강력한 실전 테크닉입니다.

06 · AUTOMATION

스킬과 루틴

자동화를 이루는 두 개의 블록. **스킬(Skill)**은 어떤 일을 '어떻게' 수행하는가를 정의합니다. **루틴(Routine)**은 그 일이 '언제' 일어나는가를 결정합니다. 스트리밍에서 살아남은 거의 모든 지속 가능한 워크플로우는 바로 이 순서대로 구축되었으며, 사람이 최소 한 번 이상 직접 손으로 해보기 전에는 결코 자동화하지 않았습니다.

1. 직접 손으로 해보기



2. 안정적 결과 도출



3. 스킬로 저장하기



4. 루틴 등록하기

스킬이 생성되는 3가지 방식

스킬은 전역(Global)입니다. 한 봇에게 가르치면 모든 봇이 공유합니다.

- **직접 시연하기 (By demonstration)**. "태스크 가르치기(Teach a task)"를 누르고 작업을 직접 한 번 수행하면, 봇이 화면 녹화를 재사용 가능한 스킬로 변환합니다. 슬라이드 애니메이션 제작과 경쟁사 블로그 스캔을 가르칠 때 실제로 쓰였습니다.
- **오류 교정하기 (By correction)**. 가장 가치 있는 방식입니다 (아래 쿼아웃 참조).
- **직접 작성하기 (By writing it)**. 프로세스에 복잡한 조건 분기, 예외 처리, 사람의 승인 단계가 포함된 경우 한 번의 시연만으로는 봇이 이를 유추할 수 없습니다. 이때는 손으로 직접 로직을 작성해야 합니다.

루틴 정의 시 반드시 필요한 항목

실행 빈도는 비용 누수의 1순위 주범입니다 — 섹션 12 참조.

- 어떤 봇이 해당 루틴을 소유하고 실행하는가
- 실행 스케줄 및 타임존 — 또는 웹훅/이벤트 트리거
- 입력 데이터가 유입되는 출처
- 기대하는 최종 산출물 포맷
- 사람의 승인이 필요한 명확한 경계선
- 실패 시 처리 절차(Fallback)
- 보고할 내용이 없을 때 취할 조치 — 아무 변경 사항이 없다면 침묵을 지키도록 명시하십시오(No-op silence).

*루틴 빈도는 돈이 조용히 사라지는 곳입니다. 정기 스케줄보다는 이벤트 트리거를 선호하고, 15분마다 실행하기보다는 하루 1~2회를 기본값으로 삼으십시오.



사건의 디테일이 아니라, 교정의 원칙을 기록하라

로산(Roshan)의 원칙: "에이전트가 잘못 생각하는 것을 목격할 때마다, 매번 프롬프트를 뽀질하기보다 이를 영구히 교정할 스킬을 만들 좋은 기회로 삼아야 합니다." 링 시(Ling Shi)의 표현은 더 날카롭습니다. 배수관을 매번 뚫지 말고 한 단계 더 깊은 근본 원인으로 내려가라는 것입니다. 하지만 반대편에도 함정이 있습니다. 특정 세션의 실패에 대응해 규칙을 쓸 때, "에이전트들은 그 세션의 사소한 디테일까지 전부 규칙에 집어넣는 경향이 있으며, 그러면 규칙이 지나치게 과적합되어 재사용성이 떨어집니다." 사건의 스토리는 지우고, 일반적인 원칙만 남기십시오.

07 · THE SOFTWARE FACTORY

433개의 PR이 머지된 비결

'소프트웨어 팩토리(Software Factory)'는 헤드라인을 장식한 수치를 만들어낸 핵심 패턴입니다. 개발팀은 이 용어를 썩 좋아하지 않았지만 ("솔직히 팩토리라는 단어 자체를 별로 안 좋아해요"), 결국 이 이름으로 정착되었습니다. 화려한 브랜딩을 건너내면 본질은 단순한 루프입니다: 작업자 에이전트가 작은 변경 사항을 작성하고, 검증자 에이전트가 앱을 직접 실행해 깨뜨려 보려 시도하며, 그 검증을 통과해야만 코드가 머지되는 구조입니다.

핵심 개발 루프 (THE LOOP)

- 1** 계획을 여러 단계(Phase)로 분할. 로렌의 Pstack 플러그인과 '포테이토 모드(Potato Mode)' 스킬이 이를 수행: `/potato mode + "이 계획을 완전 자율(Full Autopilot)로 진행해 줘."`
- 2** 구현 에이전트들이 작은 단위의 PR을 작성. Cursor 클라우드 에이전트들이 각자 독립된 머신에서 범위가 제한된 변경 작업을 병렬 수행.
- 3** 검증 에이전트들이 앱을 띄우고 퍼징(Fuzzing) 테스트 수행. '스웜(Swarm)' 스킬이 여러 에이전트를 소환하여 실제로 앱 화면을 클릭하며 결함을 추적.
- 4** CI 그린 통과 시 자동 머지. 사람이 코드 diff를 한 줄씩 읽는 대신, 자동 검증 통과 여부가 머지의 최종 관문이 됨.
- 5** 플레이테스터 봇이 엔드투엔드로 전체 게임을 플레이. 'Chrome' 봇이 CI를 통과한 PR들을 감시하고, 실제 게임을 플레이하여 이상이 없을 때 비로소 배포 승인.

자율주행의 4단계 사다리

실제 프로덕션 환경에 배포되자 로렌은 신중하게 물러섰습니다.

- **조사 전용 (Investigate only)** — "아직 PR 열지 말고, 무슨 일이 일어나고 있는지 네 생각을 먼저 가져와."
- **드래프트 (Draft)** — PR을 열어두고 사람의 검토를 대기.
- **오토파일럿 (Autopilot)** — 구현과 검증을 수행하고, 테스트 통과 시 머지.
- **완전 자율주행 (Full autopilot)** — 계획 수립, 단계 분할, 구현, 검증, 머지까지 전 과정 무인 수행.

아키텍트(Architect) 단계

서로 다른 4개 모델 병렬 합의

- Pstack의 'architect' 스킬은 하나의 문제를 **4개의 서로 다른 LLM 모델**에 병렬로 던진 후, 판정(Judging) 단계를 거쳐 최선의 계획을 선정하거나 병합합니다.
- 모델마다 서로 다른 제약 조건을 발견해 내므로 중요한 아키텍처 결정 시 충분한 가치가 있습니다.
- 초기 프로토타입 단계에서는 의도적으로 건너뛰었습니다: "지금 단계에선 솔직히 아키텍처 따위는 아무래도 상관없으니까요."



솔직하게 짚고 넘어가야 할 점

로렌은 게임 개발 과정에 대해 솔직히 고백했습니다: "솔직히 말씀드리면, 전 코드 자체는 거의 쳐다보지도 않았어요. 그냥 포테이토 모드와 Pstack을 돌렸을 뿐이죠." 반면 실제 Grok Bot 제품 자체를 만드는 에릭(Eric)은 자신이 본래 프로덕션 코드베이스에서는 절대 이런 식으로 작업하지 않는다고 분명히 선을 그었습니다 — 그는 모든 PR을 꼼꼼히 읽고 안티패턴 규칙을 엄격히 적용합니다. 팩토리의 공격성은 **위험 반경(Blast Radius)에 비례해 조절되어야 하며**, 72시간짜리 데모 게임은 위험 반경이 극도로 작은 특수한 사례였습니다.

08 · VERIFICATION

검증이 성패의 전부다

이번 3일간의 기록에서 단 하나의 지침만 가져가야 한다면, 바로 이 장을 가져가십시오. 봇을 이용해 실제 의미 있는 성과를 낸 모든 발표자가 이 원칙을 강조했으며, 아직 검증 루프를 구축하지 못한 이들은 그것이 최대의 병목이라고 털어놓았습니다.

“검증은 정말이지 너무나 중요합니다. 봇들이 실제로 코드를 실행해 보고, 스크린샷을 찍어보게 만들어야 합니다. 그래야 봇이 문제를 진짜로 이해하고 있다는 확신을 가질 수 있습니다.”

로렌(Lauren) · Day 3

검증 스킬에 실제로 들어가야 하는 것

- **에이전트가 직접 제어할 수 있는 CLI.** 매번 일회성 스크립트를 새로 작성하게 하지 마십시오 — 토큰이 낭비되고 재현성이 떨어집니다. 로렌: "봇들이 앱과 결정론적으로 상호작용할 수 있게 해주는 표준 CLI나 도구가 필요합니다. 저는 표준화된 도구 세트를 쥐여주는 쪽을 선호합니다."
- **피처 맵 (Feature Map).** 앱의 주요 기능과 흐름을 머신이 읽을 수 있는 형태로 정리해 두어, 에이전트가 헤매지 않고 테스트할 화면으로 곧장 찾아갈 수 있게 해야 합니다. Thursday Arena는 한 걸음 더 나아가 /rules 페이지에 llms.txt 스타일의 엔드포인트를 제공하여 에이전트가 게임 규칙을 직접 읽게 했습니다.

팀이 철저하게 강제한 3가지 규칙

- **1. 수정하기 전에 버그부터 먼저 재현하라.** "버그를 실제로 재현할 수 있을 때에만 에이전트가 문제를 제대로 이해했다고 신뢰할 수 있습니다." 상시 프롬프트: "코드를 작성하기 전에, 앱을 띄우고 정확한 버그와 동작을 확인한 다음 진행하라."
- **2. 증거를 PR에 반드시 첨부하라.** UI 변경 시 스크린샷이나 녹화 영상, 백엔드 변경 시 성능 메트릭을 필수로 첨부하도록 명문화했습니다. 클라우드 에이전트는 자체 실행 화면을 영상으로 녹화해 첨부할 수 있습니다.
- **3. 스킬에 이름을 붙이고 이름으로 호출하라.** 이 팀의 스킬명은 /verify cupcake이었으며, 모든 오토파일럿 지시문에서 협상 불가능한 필수 조건으로 지정되었습니다.

*검증 스킬이 없을 때 발생하는 전형적인 실패 양상은 매우 익숙합니다: 봇이 작업을 마친 뒤 "자, 이제 앱을 띄워서 여기저기 클릭해 보시고 잘 동작하는지 제게 알려주세요"라고 말하는 상황입니다. 바로 그平庸 과정에서 인간의 시간이 다 날아갑니다.



일반화 가능한 자동화 테스트 기준

무엇을 자동화할지 결정하는 에릭(Eric)의 휴리스틱: "당신의 업무 흐름을 살펴보고, '결과를 검증할 수 있는 루프'가 존재하는지 자문해 보십시오 — 그곳이 바로 봇들을 마음껏 날뛰게(Go crazy) 풀어놓을 수 있는 영역입니다." 코딩이 대표적인 사례인 이유는 테스트, 빌드, 실행 중인 앱이 기계가 읽을 수 있는 확실한 신호를 제공하기 때문입니다. 그런 신호가 없는 작업이라면, 자율성을 부여하기 전에 검증 신호부터 먼저 발명해야 합니다.

09 · PROMPT PATTERNS

실전에서 검증된 7가지 프롬프트 패턴

이것들은 알파한 프롬프트 기교가 아닙니다. 3일간 6~7명의 서로 다른 발표자들이 각자 독립적으로 자연스럽게 사용했던, 실전에서 살아남은 소수의 핵심 습관들입니다.

1. 자신의 언어로 되짚어 말하게 하기

거의 모든 발표자가 필수 사용

길거나 복잡한 지시를 내린 후, 작업을 시작하기 전에 **"이 내용을 네 말로 다시 요약해 봐"**를 덧붙입니다. 비용이 들지 않는 시점에 오해를 즉시 잡아냅니다. 2일차 발표자는 이를 "내가 가장 좋아하는 패턴"이라고 불렀습니다.

2. 막 쏟아내고, 그 뒤에 구조화하기

음성 받아쓰기(Voice dictation) 활용

마이크를 잡고 1~2분간 의식의 흐름대로 마구 이야기한 다음, 봇에게 이를 체계적으로 종합하라고 시킵니다. 나중에 다시 타이핑할 필요 없이 대화 중에 실시간으로 성장 아이디어를 포착했습니다.

3. 핵심만 추출한 뒤 추론하기

매우 긴 입력값 처리 시

장문의 음성 프롬프트가 주어졌을 때, 봇에게 먼저 핵심 사실들만 추려내게 하고, 오직 그 요약본을 바탕으로 추론을 진행하게 합니다. 두서없는 장문에서 발생하는 환각을 막는 블레이크의 비결입니다.

4. 최종 결과물부터 정의하기

암리타(Amrta)의 기법

"내가 만들고자 하는 최종 결과물은 이것이다"로 시작하여 봇이 역산하여 작업하게 합니다. 세부 단계를 일일이 나열하는 것보다 최종 산출물과 합격 기준(Acceptance Criteria)을 명시하는 편이 훨씬 낫습니다.

5. 손대기 전 조사부터

로산: "원인을 파고들어 무슨 일인지 파악해라. 아직 PR은 열지 말고, 네 분석 내용만 먼저 가져와라."

6. 중간에 개입하고 톡 치기

링 시: 봇이 궤도를 벗어날 때 중간에 방향을 틀어주는 것은 일상적입니다. 몇 분마다 상태를 묻는 것이 낫습니다.

7. 음성 메모로 직무 덤프

블레이크: 15분 음성 메모로 업무를 쏟아내고, 봇에게 "나를 위한 업무 시스템을 설계해 줘"라고 말합니다.



7가지 패턴 아래 깔려 있는 공통의 본질

이 패턴들의 본질은 하나입니다: **봇이 내 돈과 토큰을 쓰기 전에, 자신이 무엇을 해야 하는지 이해했음을 먼저 증명하게 만드는 것입니다.** 첫 출근한 유능한 신입사원에게 우리가 자연스럽게 기대하는 방식과 정확히 같습니다 — 그리고 이는 우연이 아니라, 제품 전체가 바로 이 '동료 비유' 위에 세워져 있기 때문입니다.

10 · PROMPT LIBRARY

실전 프롬프트 원문

스트리밍 현장에서 사용된 프롬프트들을 거의 손질하지 않고 날것 그대로 수록했습니다. 중요한 것은 프롬프트의 형태입니다: **목표 결과물, 제약 조건, 신뢰할 수 있는 출처, 그리고 작업 완료 시 취할 행동**이 어떻게 결합되어 있는지 주목하십시오.

비서실장(CHIEF OF STAFF) 봇 생성 프롬프트

"네 역할은 Email Ethan, Slide Sonia, Data Dan에게 연락해서 각자 지금 무슨 작업을 하고 있는지 업데이트를 받는 것이다."

→ 뒤이어 등록된 루틴: "2시간마다 팀원들에게 진행 상황을 묻고, 병목이나 차단 요소(Blocker)가 있는지 확인해라."

조용히 대기하는 노션 지식창고 봇

"이 봇의 임무는 다른 모든 봇들의 대화를 모니터링하는 것이지만, 구체적으로 호출되지 않는 한 아무것도 하지 않고 대기하는 것이다. 메시지가 네게 올 때까지 기다려라. 우리는 노션(Notion)을 선별적으로만 업데이트하길 원한다. 모든 정보를 무작정 쏟아붓고 싶지 않다. 그러니 반드시 내게 먼저 확인할 것."

— 로산(Roshan). 이 프롬프트의 핵심은 절제(Restraint) 조항입니다. 다른 곳에서는 이를 "마치 git log처럼 다루라"고 표현했습니다.

매번 소리치는 대신 한 번 정의해 두는 PO 긴급 대응 방침

"5분마다 클라우드 에이전트들을 점검하는 루틴을 설정해라. 에이전트들이 궤도를 이탈했는지 확인하라 — 예를 들어 sleep 300 같은 긴 대기 상태에 빠졌거나, 원래 목표에서 벗어났거나, 지나치게 소극적으로 변했는지. 이상을 발견하는 즉시 개입하여 주의를 환기하고 바로잡아라."

— 링 시(Ling Shi). 그의 핵심 통찰: 에이전트에게 매번 "이거 긴급해!"라고 소리치면 단계를 건너뛰고 역측을 남발합니다. 잘 정의된 정책은 그렇지 않습니다.

프로덕션 버그를 신중하게 다루는 법

"우리 ELO 시스템이나 매치메이킹이 꼬인 것 같아. ThursdayArena.com에서 게임을 할 때마다 다른 플레이어의 덱 대신 AI 플레이어와 매칭되는데, 리더보드의 ELO 점수는 요동치고 있어. 무슨 일인지 원인을 파악해 봐. 포테이토 모드를 사용하고, 조사는 Cursor 클라우드 에이전트를 써라. 아직 PR은 열지 마. 무슨 상황인지 네 분석 내용만 먼저 가져와."

— 로산. 증상, 증거, 사용할 도구, 그리고 코드 작성 전 명시적인 정지 명령까지 완벽한 구조입니다.

**안전 조항이 포함된 피드백 파이프라인 구축**

"위 링크는 유저 피드백이 들어오는 우리 슬랙 채널이야. 피드백을 살펴보고, 분류(Triage)하고, 이슈 재현을 시도한 다음 노션 데이터베이스에 티켓을 등록하는 팩토리 워크플로우를 구축하고 싶어. 매우 중요한 점은, **AI를 사용해 피드백을 검토할 때는 프롬프트 인젝션(Prompt Injection) 공격에 주의하도록 AI에게 반드시 지시해야 한다는 거야.** ... 이해한 내용을 네 말로 요약해 봐."

— 로렌. 유저가 제출한 텍스트는 신뢰할 수 없는 외부 입력입니다. 파이프라인을 구축하는 프롬프트에 이 보안 경고를 반드시 명시하십시오.

10 · PROMPT LIBRARY, CONTINUED

리서치, 개발 및 시장 진출

프로토타이핑 전문가 봇 소환하기

“Cupcake 저장소에 만든 클라이언트 앱을 위해 프론트엔드 UI 탐색을 더 해보고 싶어. 앱을 극도로 가볍게 유지하면서도 3D 애니메이션을 더 많이 추가할 수 있는 프로토타입 제작 전담 봇을 하나 띄워줄래? 이 봇은 작업을 수행하기 위해 Cursor 클라우드 에이전트를 쓸 수 있어야 해. 여러 프론트엔드 작업에 걸쳐 포테이토 모드로 에이전트 스위치를 돌릴 수 있으면 좋겠어.”

— 로렌이 닥터 에그봇에게 보낸 프롬프트. 산출물이 아니라 제약 조건("극도로 가볍게")과 실행 메커니즘을 명시한 점에 주목하십시오.

최종 산출물을 명시한 경쟁사 리서치

“샌프란시스코 최고의 레스토랑 경영자들을 전부 뽑아내고, 그중 가장 훌륭한 웹사이트 5개를 골라줘. 그다음 그 웹사이트들에서 가장 뛰어난 팝업 페이지 5개를 찾아내. 그다음 지난 6~12개월간 주요 대도시에서 진행된 최고의 팝업 레스토랑 사례 5개를 찾아줘. 그리고 그 결과들을 하나의 문서로 깔끔하게 정리해 줘.”

— 코디 산체스(Cody Sanchez). 단계마다 범위가 좁혀지는 체이닝(Chaining) 기법과 명확한 산출물 문서 지정.

요약으로 끝나지 않고 전략적 포지셔닝으로 이어지는 시장 조사

“우리 웹사이트를 분석해서 제품이 무엇이고 우리가 어떤 시장에서 플레이하는지 깊이 있게 이해해 봐. 이제 경쟁사 분석을 수행해라: 경쟁사들을 식별하고 깊이 있게 이해하며, 그들의 마케팅 웹사이트와 포지셔닝을 분석해. 그리고 가장 중요하게, 우리가 그들에 맞서 전략적, 경쟁우위적 포지셔닝을 취할 수 있는 기회와 틈새(Gaps)를 도출해 내라.”

— 조시 김(Josh Kim). 단순 정보 나열 리서치 리포트를 실질적인 전략 실행 문서로 탈바꿈시키는 마지막 문장.

명명된 스킬을 통해 에이전트 스위칭에 작업 위임하기

“이 광고 슬롯에 누구를 넣어야 할지 ICP(이상적 고객 프로필)를 찾는 걸 도와줘. 너한테 'FindMyICP'라는 스킬이 있어. ... 우리가 도출한 타깃 페르소나를 Clay MCP에 넣고, 기업 및 인물 데이터베이스를 검색해서 이 조건에 맞는 기업 10곳의 구체적인 목록을 뽑아줘.”

— 명명된 스킬 호출, 명시적인 커넥터 지정, 범위가 한정된 산출물 요구.

메모리에 영구적인 선호도 각인시키기

“앞으로 나를 부를 때 내 성과 이름을 함께 부르지 말고, 오직 내 이름(First name)으로만 부르도록 해.”

— 암리타. 한 번만 말하면 영구 메모리에 저장되어 이후 모든 대화에 지속 적용됩니다.

*이 모든 프롬프트들을 관통하는 공통 패턴: 신뢰할 수 있는 단일 출처(Source of Truth)를 명명하십시오. "모든 주장은 Sherlock 봇의 데이터를 기반으로 할 것"이라는 문구가 봇의 역할 정의에 포함되었습니다. 전문가 봇에게 어떤 동료의 말이 권위 있는 출처인지 알려줄 때, 환각은 갈 곳을 잃고 소멸합니다.

11 · ORCHESTRATION

도구가 아닌 팀을 운영하는 법

봇이 대략 5개를 넘어가기 시작하면, 사용자가 모든 메시지를 일일이 중계하고 라우팅하는 것이 시스템 최대의 병목이 됩니다. 이를 극복하기 위해 스트리밍에서 반복적으로 등장한 **4가지 오케스트레이션 패턴**이 있습니다.

1 · 비서실장 (The Chief of Staff)

- 사용자가 거의 전적으로 대화하는 단 하나의 총괄 봇; 세부 전문가들에게 업무를 라우팅하고 결과를 취합해 보고합니다.
- 새로운 봇이 생성되면 비서실장이 직접 온보딩을 진행 — 맥락과 플레이북을 전달하므로 사용자가 말을 반복할 필요가 없습니다.
- 사이먼의 원칙: 모든 하위 봇을 비서실장을 통해 생성하여, 비서실장이 각 봇의 존재 목적을 정확히 알게 하라.
- 블레이크는 중간 관리자 없이 단 한 명의 비서실장 아래 15~20명의 전문가 봇을 직속으로 운영했습니다.
- 단, 만능은 아닙니다 — 셉(Shub)은 전문가와 직접 대화하는 것을 선호한다고 밝혔습니다.

2 · 플레이북 브로드캐스트

- 하나의 봇이 노션(Notion) 등에 저장된 팀의 '살아있는 플레이북 (규정집)'을 총괄 소유합니다.
- 새로운 팀 기준이 생기면 이 봇에게 단 한 번만 말해주고, 봇이 전체 팀원에게 이를 전파합니다.
- "당신은 무엇을 해야 할지 딱 한 번만 생각하면 됩니다... 모든 엔지니어 봇이 당신이 일일이 말하지 않아도 그 규칙을 알게 됩니다."
- 표준 규칙 예시: 모든 PR은 UI의 경우 스크린샷, 백엔드의 경우 성능 지표를 필수로 첨부해야 한다.

3 · 스태프 미팅 (The Staff Meeting)

- 중요한 결정을 내릴 때 서로 다른 관점을 가진 여러 봇을 하나의 그룹 스레드에 초대하여 토론시킵니다.
- 블레이크는 봇들에게 항상 반대 의견을 내라고 지시했습니다 — "다들 찬성만 하면 부른 의미가 없으니까요."
- 비서실장이 난상토론을 종합하여 사용자에게 최종 추천안을 보고합니다.
- **경고: 그룹 채팅은 매우 수다스럽고 비쌉니다.** 일상 업무가 아니라 의사결정이 필요할 때에만 제한적으로 사용하십시오.

4 · 하위 봇 군단 (Sub-bot Army)

- 중간급 봇 하나가 컨텍스트가 가벼운 대규모 워커 봇 군단에게 대용량 배치 작업을 위임합니다.
- 사이먼의 '솔저(Soldiers)' 봇들은 공유 그룹 스레드를 통해 100~200개 기업 어카운트를 병렬로 동시 조사했습니다.
- 확장이 매우 용이하며, 산출물은 총괄 봇 한 곳으로 모여 검수를 거칩니다.
- 엔지니어링 스웬과 동일한 구조: 한 명의 오케스트레이터와 수많은 일회용 일꾼들.



아무도 완전히 해결하지 못한 현실적 고민

수많은 에이전트를 병렬로 돌리는 소감이 어떠냐는 질문에 매튜 버먼(Matthew Berman)은 이렇게 답했습니다: "그 어느 때보다 많은 일을 해내고 있습니다. **하지만 동시에, 그 어느 때보다 바쁩니다.**" 동시에 돌아가는 수십 개의 에이전트 사이를 오가며 컨텍스트를 전환하는 것은 현실적이며 아직 완벽히 해결되지 않은 인지적 비용입니다. 비서실장 패턴은 현재 우리가 취할 수 있는 최선의 해법이지만, 이는 완벽한 치료제가 아니라 고통을 줄여주는 완화책일 뿐입니다.

12 · ECONOMICS

실제 소요 비용과 비용 누수 방지

과금은 사용량 기반입니다: 봇과 나누는 대화 토큰 비용과 봇 자체의 컴퓨터 사용(Computer-use VM) 연산 비용을 지불합니다. 다음은 방송 중에 직접 언급된 실제 비용 수치들과, 참가자들이 돈을 낭비하는 것으로 목격된 4가지 핵심 누수 지점입니다.

\$20-30

세일즈 케이스 스타디 슬라이드 덱
1편 제작 (수작업 시 4~5시간
분량)

\$1-2

중간 난이도 고객 지원 티켓 1건
엔드투엔드 처리

\$0.20

단순 케이스 버킷 분류 및 배치
스크립트 적용 후 티켓당 비용

~\$1,000

봇이 더 저렴한 전기 요금제를 찾아
아낀 연간 절감액 (인간 시간
60초)

비용이 새어 나가는 4대 구멍 (THE FOUR LEAKS)

- 1 과도하게 잦은 루틴.** 압도적인 비용 누수 1위 요인. "15분마다 실행되는 루틴이 3개만 있어도 하루에 수백 건의 메시지가 발생합니다." 실행 빈도를 감사하고, 정기 스케줄보다 이벤트 트리거를 쓰며, 일상 업무는 하루 1~2회로 충분합니다.
- 2 API가 존재하는데 브라우저 자동화를 쓰는 경우.** 컴퓨터 유즈로 웹 폼을 클릭하는 것은 네이티브 API 호출보다 훨씬 비쌉니다. 셉(Shub)의 노하우: 봇이 브라우저로 작업하는 것을 한 번 지켜본 뒤, 트리거된 네트워크 요청을 분석하게 하여 다음부터는 해당 엔드포인트를 직접 쓰게 만드십시오.
- 3 방치된 그룹 채팅.** 그룹 채팅의 봇들은 서로의 말을 받아치며 끝없이 수다를 떨어 토큰을 순식간에 태웁니다. 일상 업무는 1:1 멘션 호출을 유지하십시오.
- 4 봇의 무분별한 증식 (Bot Sprawl).** 봇이 늘어날수록 더 많은 루틴이 돌고, 컨텍스트가 중복되며, 혼선이 가중됩니다. 군살 없는 슬림한 팀이 가장 경제적인 팀입니다.

실제로 조절 가능한 비용 레버

- 봇의 응답 길이(Verbosity) 조정 — 말투와 장황함의 정도가 토큰 소비에 직결됩니다.
- 더 이상 필요 없는 지난 맥락은 봇에게 잊으라고(Forget) 지시하십시오.
- Grok Bot 자체에게 현재 설정 중 비용 최적화할 부분이 있는지 진단을 요청하십시오.
- 다른 봇들의 루틴과 스킬 효율을 지속적으로 감사하고 최적화하는 전담 봇을 두십시오.

조절할 수 없는 부분

컴퓨터 유즈 자체의 연산 단가는 사용자가 직접 튜닝할 수 없습니다. 비용을 줄이는 유일한 방법은 **덜 쓰는 것**입니다: MCP 커넥터가 있다면 연결하고, 다른 대안이 도저히 없을 때에만 최후의 수단으로 브라우저 조작을 사용하십시오.

13 · APPROVALS AND BLAST RADIUS

브레이크가 있는 자율성

매튜(Matthew)의 정리는 가장 명쾌했습니다: "봇을 신뢰하고 그들에게 주도권(Agency)을 부여하십시오. 그리고 동시에 가드레일을 쳐두십시오. 무언가를 세상에 배포하기 전에 봇이 반드시 당신에게 확인을 받으러 올 것임을 알기에, 일이 파국으로 치닫지 않는다는 확신을 가질 수 있습니다."

오토 리뷰와 명시적 규칙

- 내장 분류기가 작업의 위험도를 평가하여 위험한 행동 전에는 사용자에게 질문합니다.
- 그 위에 명시적 규칙을 엮었습니다: "내 승인 없이 이메일 보내지 마라", "슬라이드 작성은 자유롭게 해도 좋다."
- 액션 타입별 토크 — 예: 프로덕션 배포는 항상 먼저 물어볼 것.
- 조심스러운 기본값: 한 봇은 품을 생성하기 전 개인정보(PII)를 수집하는지부터 스스로 검토했습니다.

자격증명과 보안 금고 (Vault)

- 보안 품을 통해 시크릿을 저장소에 보관하며, 에이전트는 절대 원본 비밀번호를 직접 보지 못합니다.
- 행사 도중 1Password 연동이 출시되어 봇이 금고를 통해 안전하게 재인증할 수 있게 되었습니다.
- 공유 템플릿에는 메모리, 설정, 스킴, 루틴만 복사되며 비밀번호나 대화 내역은 제외됩니다.
- 위임하고 싶지 않은 중요 로그인인 사용자가 직접 화면을 제어하십시오.

실전 방송에서 뼈아프게 배운 3가지 교훈

- 경쟁 요소가 있는 기능은 반드시 서버 권한(Server-authoritative)이어야 한다.** 게임 로직이 클라이언트에 남아있자 멧은 개발자 도구를 열고 실시간으로 자기 스탯을 조작했습니다. 클라이언트에 아예 관련 코드가 없어야 합니다.
- 유저가 제출한 텍스트는 전부 공격 표면(Attack Surface)이다.** 피드백 파이프라인에 클라이언트 길이 제한, 욕설 필터, 입력값 살균, LLM 가드, 레이트 리밋, 프롬프트 인젝션 방어 지침을 전부 탑재해야 했습니다.
- 자율적 버그 수정이 프로덕션을 마비시킬 수 있다.** 실제로 팩토리가 잘못 작성한 SQL 쿼리가 방송 도중 라이브 게임을 다운시켰습니다. 마이그레이션과 배포에는 아무리 자동화가 훌륭해도 사람의 승인 관문을 유지해야 합니다.



신규 봇을 위한 기어가기-걷기-뛰기 사다리 (Crawl-Walk-Run)

고객 지원에서 데이비드(David)가 보여준 단계적 접근법은 모든 분야에 적용됩니다: 처음에는 오직 문서를 읽고 요약만 하는 봇으로 시작하고, 그 다음엔 내부 메모 형태로 초안만 작성하게 하며, 신뢰가 충분히 쌓인 후에야 비로소 직접 발송하고 행동하도록 권한을 엮니다 — 이때도 고객 대면 지식창고 수정과 같이 **위험 반경(Blast Radius)이 가장 큰 행동에는 사람의 승인을 필수 요건으로 유지**합니다. "단순하게 시작하십시오. 봇을 여러분의 기존 업무 방식에 맞추려 해야지, 그 반대가 되어서는 안 됩니다."

14 · CASE STUDY

Thursday Arena, 72시간 만에 완성된 게임

공개 마켓플레이스에 등록된 실제 봇들을 캐릭터로 소환하여 희귀도, 생성된 아바타, 스탯, 고유 능력을 부여한 오토배틀러 게임입니다. 단 세 명의 인원과 봇 팀이 개발하여, 3일차 오전 10시 프로필 사진도 없는 신규 X 계정의 트윗 하나로 공식 출시되었습니다.

어떻게 빌드되었는가

- **프로토타입 우선** — 바닐라 HTML/CSS/JS, 인메모리 상태, DB 없음, 인증 없음. 유일한 질문: "이 게임 루프가 과연 재미있는가?"
- 루프의 재미가 검증된 후 React/Next.js로 전면 재작성, 클라이언트와 서버 분리.
- 백엔드는 Vercel Serverless 위의 Go, 데이터베이스는 PlanetScale, 인증은 Clerk (Sign in with X), 네트워크 경계 검증은 Zod.
- 카드 아트는 Grok Imagine 생성; 캐릭터는 스프라이트 대신 코드로 제어하여 상태를 프로그래밍 방식으로 전환.
- 초반에는 테스트 코드를 의도적으로 삭제 — "에이전트들은 일반적으로 테스트를 실패하지 않습니다" — 코드 품질 단계는 후순위로 미룸.

성과를 가른 핵심 설계 결정들

- **Pay-to-Win 전면 배제.** 수없이 논의되었으나 단호히 거절됨. 광고, 스폰서 카드, 굿즈가 채택된 수익 모델.
- **가차 없는 기능 처내기 (Ruthless cutting).** 가위바위보 상성, 복잡한 스탯 블록, 다중 전장 기획을 전부 잘라내고 가장 깔끔한 단일 루프만 남김.
- **복잡한 숫자의 은닉.** 날것의 ELO 수치 대신 티어로 대체; 구체적 스탯 대신 상대적 강도로 시각화.
- 실시간 치팅 시연 후 전면적인 **서버 권한화 (Server Authority)** 단행.
- 인간뿐만 아니라 AI 에이전트도 읽을 수 있도록 기획된 `/rules` 페이지 구축.

출시 당일 주요 지표

170

3일차 방송 시작 전 밤사이에 머지된 PR 수

1,908

출시 첫 1시간 동안 플레이된 경기 수 (당일 최고치)

~6,000

약 2,000명의 유저가 플레이한 총 공개 매치 수

#1

출시 당일 구글 검색 결과에서 게임 이름으로 1위 등극

*피드백은 세 가지 채널을 통해 하나의 슬랙 방으로 모였습니다: 인증 기반 인앱 폼, 음성 에이전트 기반 전화번호, 그리고 X 멘션. 이 채널들의 피드백을 분류하고, 버그를 재현하며, 노션 티켓으로 자동 발행하는 작업은 출시 당일 오후에 전면 자동화되었습니다.



실제 퍼널 데이터가 말하는 진실

연습 모드 플레이어의 약 8%가 정식 계정 로그인으로 전환되었으며, 트래픽은 모바일과 데스크톱이 거의 5:5로 양분되었습니다(iOS 약 44%). 접수된 피드백 중 가장 큰 비중을 차지한 것은 모바일 레이아웃 이슈였습니다 — **전체 피드백의 71%가 버그 리포트였고, 칭찬은 16%에 불과했습니다.** 팀은 이 모바일 피드백에 대응해 당일 오후 즉시 반응형 대응 패치를 배포했습니다. 초기 유입의 거의 전부는 라이브 스트리밍 시청자로부터 나왔으며, 이는 모든 수치에 감안해야 할 전제조건입니다.

15 · THE ROSTER

팀에 투입된 모든 봇들의 명단

Thursday Arena 제작에 실제로 투입된 봇 팀의 전체 명단입니다. 하나의 모범 템플릿으로 읽어볼 가치가 있습니다: **코드를 직접 작성하는 봇은 극소수이며, 대다수의 봇이 검증, 라우팅, 피드백 수집을 위해 존재한다는 점에 주목하십시오.**

Dr. Eggbot · 메타 / 봇 팩토리

다른 모든 봇을 생성하고 감사. 디스크립션을 작성하고, 이름을 짓고, "우리 병목이 어디지?"에 답함. 3일간 가장 많이 쓰인 봇.

Cupcake Eng · 엔지니어링 오케스트레이터

포테이토 모드와 클라우드 에이전트를 가동하고 결과를 감독 및 검증하여 엔지니어링 결과물을 총괄 책임짐.

Chrome / Play · 플레이테스터

CI 테스트를 통과한 PR을 감시하고, 코드가 머지되기 전에 실제 브라우저로 게임 전체를 직접 플레이하여 검증.

Hashbrown · 검증관 (Validation)

오토파일럿이 수정 작업에 들어가기 전에 Crumble의 버그 분류가 정확한지 재확인하는, '검증자를 검증하는 봇'.

Whisk · Crit · 게임 디자인 & 비평

디자이너 겸 비평가. 출시 당일 아침 "게임이 너무 어렵다"는 Crit의 혹평이 출시 직전 밸런스 패치를 이끌어냄.

Ping · 슬랙 리스너

슬랙의 @멘션을 감시하고 진행 상황을 노션 보드에 실시간 기록. 두 사람이 각자 따로 만들었는데 에그봇이 똑같이 지어준 이름.

Vincent · Cerebro · 그로스 마케팅

노션 플레이북에 우선순위별 그로스 아이디어를 기록하고, FindMyICP 스킴을 통해 아웃바운드 타겟 파이프라인 리서치 수행.

Steve · 비서실장 / 오케스트레이터

모든 업무가 통과하는 관문. 팀원들이 혼한 "Chief of Staff"라는 이름을 서로 놀린 후 친근한 이름으로 개명.

Bake · 파운딩 엔지니어

PR 활동을 모니터링하고 머지하며, 버그 수정을 위해 클라우드 에이전트를 소환. 데모 도중 음성으로 호출되어 실시간 머지 수행.

Crumble · 버그 분류 (Triage)

플레이테스터 봇과 협력하여 유저가 보고한 버그를 직접 재현하고, 확인된 이슈만 노션에 정식 등록.

Comment Sicko · 코드 위생 관리

불필요한 코드 주석을 삭제. 에이전트들이 근본 원인을 고치는 대신 변명을 주석으로 남기는 악습을 막아 유지보수성을 지킴.

Glow · Tone · 3D & 사운드

프론트엔드 애니메이션 프로토타이핑, Strudel과 Suno를 활용한 배경음악 작곡 후 팀이 들을 수 있도록 노션에 업로드.

Data Scientist · 데이터 리포팅

가입자 수, 기능 사용량, 버그 리포트 대시보드를 정기 생성. 출시 당일에는 15분 간격으로 모니터링 보고.

*참고로 팀의 명명 규칙은 전부 감자(Potato)나 베이킹(Baking) 테마였으며, 머지된 PR은 내부적으로 "MASH"라고 불렀습니다. 닥터 에그봇과의 작명 세션에서 시작된 유행어가 실제 노션 프로세스 언어로 정착된 것입니다. 유치해 보이지만 팀이 작업에 대해 훨씬 즐겁게 소통하도록 만들었습니다.

16 · FROM THE FIELD

실무 현장의 6가지 봇 조합

3일간 진행된 실무 워크숍 세션들은 가장 직접적으로 복사해 쓸 수 있는 보물창고였습니다. 각 발표자가 실제로 운영하고 있는 현장 구성입니다.

포스트세일즈 (Post-sales)

블레이크(Blake) · AI 배포 매니저 (AI Deployment Manager)

"거스(Gus)는 제 최고의 친구입니다." 블레이크는 단 하나의 총괄 봇과만 대화하며, 그 아래 15~20명의 전문가 봇이 중간 관리자 없이 평평하게 일합니다.

팀 구성 (The Team)

- **Gus** — 비서실장이자 유일한 소통 창구
- **Frankie** — 미팅 후속 조치(Follow-up) 전담
- **Wally** — 타깃 청중별 보이스와 톤 조율
- **Trudy** — 내부 지식 및 진실의 단일 출처
- 고객 계정당 1개의 전담 봇이 해당 고객의 전체 컨텍스트를 완벽히 보유

핵심 업무 습관 (The Habits)

- "Harbor 고객 건 어디까지 진행됐지?"라고 물으면 리스크, 블로커, 열린 약속, 다음 액션 아이템 — 그리고 필수 우주 유머를 함께 반환.
- 주간 자기개선 스캔: 자동화할 수 있는 작업을 감사하고, 봇 초안과 블레이크의 최종본 diff를 비교해 봇의 말투를 교정.
- 주당 제안 건수를 최대 1개로 제한 — 스팸 방지.

"여러분의 Grok Bot을 가로막고 있는 유일한 것은, 무엇이 실제로 가능한지에 대한 여러분 자신의 상상력 한계입니다."

마케팅 (Marketing)

조시 김(Josh Kim) · 6인조 캠페인 봇 팀

마지막에 채용된 봇이 앞선 다른 봇들을 자동화하는 구조: 시장 조사 → 프로덕트 마케터 → 웹사이트 운영 → 퍼포먼스 마케터 → 마케팅 분석 봇. 그리고 6번째 봇인 프로젝트 매니저가 전체 캠페인을 엔드투엔드로 지휘합니다.

역할 정의 노하우

- "채용 공고의 직무 기술서를 작성하는 것과 거의 같습니다" — 봇마다 철저하게 분리된 전용 스왐레인 부여.
- 포지셔닝 기획서는 실제 동료와 검토하듯 구글 닥스 댓글 피드백 루프를 거쳐 완성.

조시의 실전 조언

- 슬랙과 이메일을 초기에 연결한 뒤 "나를 위해 무엇을 해줄 수 있니?"라고 물어보십시오.
- "당신의 봇에 투자하십시오" — 피드백과 컨텍스트는 실제 동료와 마찬가지로 복리로 쌓여갑니다.

16 · FROM THE FIELD, CONTINUED

영업, 고객지원, 아웃바운드, 개인 생산성

영업 (Sales) · 크리스타(Krista, AE)

복잡한 툴 더미를 관통하는 오케스트레이션 레이어: Olive(비서실장), PG(아웃바운드 발굴), Echo(Gong과 Granola 트랜스크립트 기반 실시간 미팅 텍스트 큐레이션), 고객 전문가 및 엔지니어 봇. Notion을 마스터 레코드로 삼고 Salesforce, Slack, Databricks를 가로지름 — "데이터를 한곳에 깔끔하게 유지하는 회사는 세상에 없습니다."

차이를 만든 요점: 자신의 센트(Sent) 메일함, 슬랙, X 활동 내역으로 봇을 학습시켜 뻘한 AI 말투를 완벽히 제거. 매일 쓰는 도구부터 연결하고, "웹세미나 대신 시청하고 이메일 초안 작성해 놔"처럼 단순 리서처가 아닌 직접 행동하는 실행자(Doer)로 대우.

고객 지원 (Support) · 데이비드(David, User Ops)

4개의 봇과 정량 평가(Evals) 테이블: Build(인프라), Reply(티켓 및 슬랙 응대), Alert(이탈 위험 및 계정 잠금 감지), Tune(자기 개선 및 지식창고 업데이트). 티켓 시스템, Stripe, Notion, Slack을 실시간 연동: 단순 리셋 자동 처리, 복잡한 SSO 장애는 낮은 신뢰도 경고와 함께 사람에게 안전하게 에스컬레이션, 환불 정책에 따른 정당한 거절, 지식창고 공백 식별 후 사람 승인 요청.

인프라 및 가드레일: 평가 실행 결과 테이블과 실행별 상세 추적(Trace) 테이블 분리. PR 브랜치 단위 평가 실행. 비기술자를 위해 낯것의 설정 대신 템플릿을 제공하고, 지식창고 수정은 코드 오너와의 PR 리뷰를 거치도록 하여 git 프리미티브를 안전장치로 재활용.

아웃바운드 및 개인 생산성 · 사이먼(SDR) & 카렌 청(크리에이터)

사이먼의 아웃바운드 군단: 비서실장, Shakespeare(이메일 톤앤매너 전담), 저비용 군단 봇들을 지휘하는 웹 검색 봇, 고객 봇 — 매일 50개 신규 리드 발굴, 아침마다 상위 5개 집중 공략. "단 하나의 이메일도 템플릿처럼 보여서는 안 됩니다." 기능별 색상 코딩.

카렌 청의 소싱 규칙: 극단적으로 단순하고 지루한 단일 목적 봇 12개 운영 — 택배 추적, 재입고 알림, 그리고 집안 와이파이에서 프린터를 알아서 찾아 아침 뉴스를 자동 인쇄하는 모닝 페이퍼 봇. "일상에서 느끼는 문제나 불편을 찾아서, 그냥 해결해 버리세요."

"이제는 바이브 코딩(Vibe Coding)의 시대가 아닙니다. 그냥 바이브(Vibing) 그 자체입니다."

17 · FAILURE LOG

라이브 방송 중에 터진 사고들

편집되지 않은 생방송은 잔인할 만큼 정직합니다. 수많은 시청자가 지켜보는 가운데 실제로 발생했던 실패들과, 그 실패가 만들어낸 영구적인 원칙들입니다.

에이전트가 자신만의 규칙에 과적합됨

Day 2

한 번의 장애를 겪은 후 작성된 봇 디스크립션이 그 사건을 영구적인 정체성으로 하드코딩함.

규칙: 실패 세션으로부터 규칙을 만들 때는 사건의 스토리를 제거하고 일반 원칙만 남겨라.

쉬머(Shimmer) 이펙트가 카드 전체를 뒤덮음

Day 2

희귀도를 나타내는 CSS 반짝임 효과가 너무 과하게 적용되어 모든 카드가 똑같이 보임. 봇이 목적이 아니라 효과 자체에만 매몰됨.

규칙: 시각적 기교가 아니라 차별화된 결과 목적을 명시하라.

실제 랜딩 페이지에 가짜 데이터가 노출됨

Day 2

에이전트가 실제 마켓플레이스를 조화하는 대신 그럴듯한 가짜 예시 봇을 날조하고 내부 프로토타이핑 용어를 카피에 유출시킴.

규칙: 신뢰할 수 있는 데이터 출처를 지정하고 '실제로 직접 찾아볼 것'을 명시하라.

3D가 아닌 2.5D 프로토타입 생성

Day 2

3D 애니메이션을 요구받았으나 3D 라이브러리를 쓰라는 지시가 없었기에 2.5D를 만들어냄.

규칙: 잘 알려진 라이브러리가 존재하는 작업이라면 라이브러리 이름을 명시하라.

개발자 도구를 통한 실시간 치팅

Day 2

게임 로직이 클라이언트에 남아있어 브라우저 콘솔 창에서 스탯을 실시간으로 조작함.

규칙: 경쟁이 있는 모든 기능은 서버 권한이어야 하며, 피쳐 플래그는 보안이 아니다.

봇이 팀의 배포 정책을 무시함

Day 1

팀이 main 브랜치에 직접 머지하기로 합의한 단계에서 봇이 멋대로 PR을 열어버림.

규칙: 말하지 않은 암묵적 규칙은 규칙이 아니다. 봇이 읽을 수 있는 곳에 명문화하라.

VM에서 로그인 세션이 끊김

Day 2

로컬 브라우저와 달리 VM 브라우저 세션이 자주 풀려 카렌 청의 자동화가 중단됨.

규칙: 무인 자동화가 필요한 작업에는 브라우저 세션보다 커넥터(API/MCP)를 우선하라.

팩토리가 프로덕션 서버를 다운시킴

Day 3

자율 버그 수정 루프가 잘못된 SQL 쿼리를 날려 실시간 게임 서버를 마비시킴.

규칙: 아무리 루프가 뛰어나도 마이그레이션과 프로덕션 배포에는 사람의 승인 관문을 유지하라.

화면에 실제 라이브 API 토큰 노출

Day 3

몇 초 만에 감지되어 폐기되었으나 방송 화면에 노출됨.

규칙: 시크릿 관리에서 가장 취약한 연결고리는 보안 볼트가 아니라 인간이다.

출시 직후 피드백 파이프라인에 스팸 폭탄

Day 3

출시 몇 시간 만에 발생. 20자 최소 길이, 욕설 필터링, 살균, 모델 가드, 레이트 리밋을 사후에 급조해야 했음.

규칙:품을 배포할 때 모더레이션(검증) 로직을 사후가 아니라 반드시 함께 배포하라.

18 · LIMITS AND ROADMAP

아직 동작하지 않는 것들

생방송 중, 주로 Q&A 세션에서 가감 없이 솔직하게 밝혀진 내용들입니다. 아직 존재하지도 않는 기능 위에 헛된 워크플로우를 설계하는 우를 범하지 않기 위해 반드시 알아두어야 합니다.

현재의 명확한 기술적 한계

- **오직 Linux만 지원.** Linux와 호환되지 않거나 MCP로 노출되지 않는 도구는 절대 사용할 수 없습니다 (Q&A에서 단호하게 "No"로 답변).
- **캡차(CAPTCHA)가 봇을 차단함.** 마땅한 범용 우회책이 없으며, 감지를 회피하려 애쓰기보다 봇이 가지 말아야 할 사이트를 차단하는 것이 공식 권장사항입니다.
- **계정 간(Cross-account) 봇 메시징 불가.** 내 봇이 다른 사용자의 봇과 직접 대화할 수 없습니다. "매우 멋진 유즈케이스이며 기대하고 있지만" 현재는 미지원입니다.
- **VM 세션 만료 문제.** 브라우저 로그인인 로컬 세션과 무관하게 주기적으로 만료되어 무인 자동화가 중단될 수 있습니다.
- **타 에이전트 스택에서의 마이그레이션 경로 부재.** 템플릿을 불러오거나 기존 컨텍스트를 지정하는 것 외에 공식적인 마이그레이션 도구는 없습니다.

곧 출시될 것으로 예고된 기능들

- **양방향 음성 모드 (Two-way voice mode)** — 3일차 방송에서 실시간 공개되었으며 순차 롤아웃 예정. 음성 통화로 봇에게 PR 검토 및 머지를 지시하는 시연 완료.
- **멀티플레이어 (Multiplayer)** — 사람과 여러 봇이 한 대화방에 참여. 현재는 슬랙에서 봇을 @태그하는 방식으로만 가능. 방송 중 가장 많은 요청을 받은 기능.
- **다중 머신 관리 (Multi-machine)** — 여러 VM을 동시에 다룰 때 봇이 혼란을 겪는 문제로, 집중적으로 개선 작업 진행 중.
- **직무 기반 맞춤형 온보딩** — 신규 유저에게 직무를 묻고 그에 맞는 맞춤형 스타터 템플릿 세트를 제공.
- **컴퓨터 유즈 속도 개선** — "가장 뜨거운 관심사"로, 향후 몇 주에 걸쳐 비약적인 속도 개선이 예정되어 있음.

*수치와 기능은 스트리밍 당시 기준이며 지속적으로 업데이트됩니다. 한계 섹션은 불변의 스펙이 아니라 현재의 윤곽으로 받아들이고, 중요한 의사결정 전 최신 공식 문서를 반드시 확인하십시오.



아무도 뽀족한 답을 내놓지 못한 근본적 난제

결정론성(Determinism). LLM 모델은 근본적으로 비결정론적이며, 이는 봇이 일관된 정책을 집행해야 하는 순간 치명적인 문제가 됩니다. 제시된 해결책은 마법 같은 프롬프트가 아니라 구조적 접근이었습니다: 봇에게 코드(의사결정 트리나 검증 함수)를 직접 작성하게 하고, 정책 판단이 필요할 때마다 매번 새롭게 추론하는 대신 해당 코드를 호출하게 만드는 것입니다. 이는 검증 CLI의 철학과 일맥상통합니다: 같은 질문에 언제나 같은 답이 나와야 하는 곳이라면, 주관적 판단을 결정론적 도구로 대체하십시오.

19 · THE RETROSPECTIVE

스트리밍 말미에 인정한 솔직한 이야기

3일차 마지막 30분은 72시간 전체 스트리밍 통틀어 가장 값진 시간이었습니다. 팀이 화려한 데모를 멈추고, 자신들이 무엇을 겪었는지 냉정하게 평가하기 시작했기 때문입니다.

“아무리 무제한의 AI 역량을 쥐고 있다 한들, 무언가를 만들 때 가장 어려운 부분은 여전히 ‘사람들의 관심을 이끌어내는 일’입니다.”

로산(Roshan) · 마무리 회고 세션

팀이 도출한 5가지 핵심 결론

- 1 배포(유통) 파워가 유일한 성공 이유였다.** 제품 자체의 본질적 매력 때문이 아니라, 실시간 라이브 스트리밍의 화제성 덕분에 유저 유입이 일어났습니다. 14장의 모든 성공 지표에는 이 전제조건이 붙습니다.
- 2 이미 잘 알고 있는 영역에서 비즈니스를 시작하라.** "모델을 잘 구슬려 돈을 벌어보겠다"는 순진한 접근은 처참히 실패했습니다. 하루 종일 돌아간 전담 수익화 에이전트의 매출은 정확히 \$0였습니다.
- 3 아이디어에 합의하는 것이 제품을 만드는 것보다 훨씬 어렵다.** 첫날 하루 종일 4번의 피벗을 거치며 완성된 제품은 0개였습니다. 개발팀의 발목을 잡은 것은 코딩 소프트웨어가 아니었습니다.
- 4 절제(Restraint)는 이제 필수적인 엔지니어링 역량이다.** 로렌: "이제는 누구나 무엇이든 만들 수 있습니다 — 아무도 안 쓸 100만 개 기능의 조잡한 제품까지도요. 이제는 무엇을 만들지 않을지 절제하는 규율이 필수입니다." 수많은 게임 메커니즘을 과감히 쳐냈기에 비로소 출시할 수 있었습니다.
- 5 문제 발견은 온전히 인간의 몫으로 남는다.** "인간인 당신이 직접 현장으로 나가 유기적으로 진짜 문제들을 발견해야 합니다. 일단 해결책을 찾아내고 나면, 그것을 봇으로 만들어 영원히 자율 실행되게 둘 수 있습니다."

봇들이 진정으로 탁월했던 영역

- 팀원 누구도 몰랐던 도메인의 지식 공백 메우기
- 사람이 하기 싫어하는 반복적이고 지루한 검증 작업
- 인간 두뇌 용량을 초과하는 3일차 프로젝트 맥락 유지
- 말로 던진 아이디어를 정렬된 목록으로 즉각 변환

여전히 인간의 몫으로 남은 영역

- 애초에 무엇이 만들 가치가 있는지를 결정하는 일
- 새로운 기능 추가 요구에 단호하게 "No"라고 거절하는 일
- 게임이 아직 재미없다는 사실을 본능적으로 알아차리는 감각
- 인간 대 인간의 신뢰 구축, 관계 형성, 모호한 정성적 판단

*흠쳐 올 만한 가치가 있는 로산의 셀프 체크 질문: 주기적으로 "내가 지금 루프에 너무 깊이 개입해 있는가? 이 과정을 어떻게 더 자율화할 수 있을까?"라고 자문하십시오 — 그리고 봇에게 이 질문을 주기적으로 던지는 루틴을 걸어두십시오.

20 · START HERE

첫 번째 주 실천 체크리스트

앞선 단계가 다음 단계를 더 쉽고 저렴하게 만들도록 순서대로 정렬되었습니다. 무엇이 가장 중요하냐는 질문에 로렌이 내놓은 우선순위는 일반적인 예상을 깨뜨렸습니다: **"Grok Bot에서 가장 중요한 것은 봇을 세팅하는 게 아닙니다. 이미 쓰고 있는 다양한 도구와 플러그인을 연결하는 것입니다."**

- 1 매일 쓰는 기존 툴 스택부터 연결하라.** 봇을 만들기 전 먼저 하십시오. Slack, 이메일, 캘린더, 이슈 트래커, 코드 저장소. 커넥터는 속도, 비용, 신뢰성 모든 면에서 브라우저를 압도합니다.
- 2 15분짜리 음성 메모를 녹음하라.** 내가 누구인지, 내 업무가 무엇인지, 지금 어디가 고장 나 있고 무엇을 자동화하고 싶은지. 그 트랜스크립트를 첫 봇에게 던지고 시스템 설계를 맡기십시오.
- 3 하루 중 가장 짜증 나는 작업 하나를 골라라.** 암리타와 카렌 청의 공통 조언: 가장 화려해 보이는 일이 아니라, 가장 짜증 나고 귀찮은 작업부터 고르십시오.
- 4 단 하나의 좁은 전문가 봇을 만들고 읽기 전용으로 시작하라.** 먼저 문서를 읽고 요약하게 하고, 내부 메모로 초안만 쓰게 하십시오. 충분히 신뢰가 쌓인 뒤에야 행동 권한을 부여하십시오.
- 5 봇이 지켜보는 가운데 작업을 직접 손으로 한 번 수행하라.** "태스크 가르치기(Teach a task)"를 쓰십시오. 그런 다음 판단 로직, 에러 처리, 승인 단계를 손으로 덧붙이십시오.
- 6 두 번째 봇을 만들기 전에 검증 루프부터 구축하라.** 아무리 조잡해도 상관없습니다. 봇이 자신의 작업물을 실제 환경과 대조해 스스로 검증할 수 있다면 이후 모든 작업이 빨라집니다.
- 7 이제야 비로소 비서실장(Chief of Staff) 봇을 도입하라.** 그리고 이후의 모든 추가 봇은 비서실장을 통해 생성하십시오. 그래야 비서실장이 각 봇의 쓰임새를 온전히 이해합니다.
- 8 교정 내용을 일반 원칙으로 축적하라.** 봇이 잘못 생각할 때마다, 특정 사건이 아닌 일반화된 원칙을 작성하십시오. 이 배움의 누적이 1주 차와 3개월 차의 격차를 만듭니다.
- 9 금요일마다 등록된 루틴들을 감사하라.** 빈번한 실행 주기는 돈이 새어 나가는 주범입니다. 정기 스케줄로 돌아가는 것 중 이벤트 트리거로 바꿀 수 있는 것은 전부 전환하십시오.

“오늘 하지 않을 이유가 있는가? (Why not today?)”

로산의 고정 슬랙 상태 메시지가자, 이 가이드를 맺기에 더없이 완벽한 한마디



그리고 절대로 하지 말아야 할 단 한 가지

재미있다는 이유만으로 봇을 마구 찍어내지 마십시오. 만드는 게 재미있다는 것, 바로 그것이 문제입니다. 새 봇을 뽑을 때마다 컨텍스트, 루틴, 주의 집중 비용이 청구되며, 방송에서 가장 큰 봇 팀을 굴렸던 이들일수록 팀을 작게 유지해야 한다고 가장 격렬하게 목소리를 높였습니다. 봇을 더 만들고 싶은 충동이 들 때마다, 기존 전문가 봇에게 새 스킬을 가르치는 것으로 해결할 수 없는지 먼저 자문하십시오.